Read this first. A worked example of a final architecture document (use case: energy-aware ventilation & climate control), written to the **grade-5 standard**, choices justified against alternatives, requirements traced to validation, the system reasoned about as a whole, and evaluation backed by measured evidence and honest critique. It is *not* a recommended or canonical architecture: simpler or more complex solutions can earn the highest grade if equally well justified, tested, and evaluated. **Copying the architecture, technologies, or text does not help**, in the oral examination each author must explain and defend the design decisions independently.

Energy-Aware Ventilation & Climate Control

D7065E, Architecture Document (worked example, targeting grade 5)

Example team: A. Andersson & B. Berg

Contents

1. Summary

This system keeps the rooms of a building comfortable and healthy while using as little energy as possible. It senses temperature, CO_2, and occupancy per room; an **autonomous service** decides how much to ventilate and heat each zone; and actuators drive the HVAC setpoint and ventilation dampers. The building itself, its rooms, sensors, actuators, and 3D view, is provided by **BuildSim**, which holds the shared building state; this project provides the physics, the intelligence, and the control around it. The guiding principle is simple and is the thread through every later decision: **ventilate and heat where and when people actually are, and ease off everywhere else.**

Field	Value
Project title	Energy-Aware Ventilation & Climate Control
Team members	A. Andersson, B. Berg
Use case	Indoor air quality + thermal comfort at minimum energy
Repository	`github.com/example-team/d7065e-ventilation`
Version / date	v1.6 / 2026-09-24

Changes since the proposal. Two design changes were made *because the evidence demanded it*, which is the point of building iteratively. First, the proposal sent readings straight to the autonomous service over REST; once a dashboard and historical storage also needed the same readings, the design moved to an **MQTT broker** so each sensor publishes once and every consumer subscribes independently. Second, the proposal planned a machine-learning occupancy predictor; on two weeks of simulated data it gave no measurable benefit over a transparent **rolling estimate with a time-of-day prior**, so the autonomous service was kept rule-based and spent the time saved on data-quality handling and testing. Both decisions are revisited in Section 7.

2. Use case and building context (BuildSim)

The building. The system controls a multi-room office floor modelled in **BuildSim**. BuildSim is the *shared state* of the cyber-physical system: it owns the building model (rooms, levels, equipment), exposes a **REST API** to register devices and to read and write their state, and streams live state to a **3D web viewer** over WebSocket. BuildSim is not modified; it is instrumented. This separation is deliberate, it is exactly the seam between the *physical half* of a CPS (the building, behind BuildSim's API) and the *cyber half* (the project's processes), and keeping that seam clean is what would let the same software later drive a real building instead of a simulated one.

How the system connects to BuildSim. Each of the project's processes talks to BuildSim through its API, and nothing else touches the building state directly:

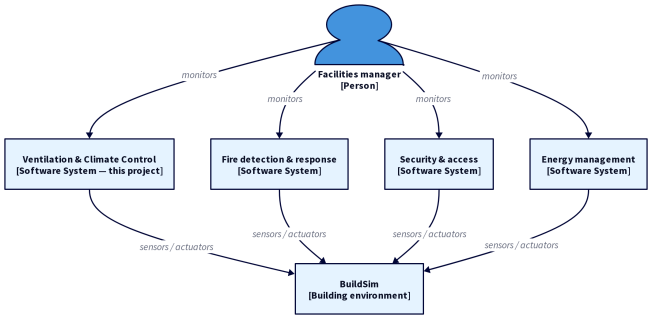
- **Sensor processes** read their sensor's current value from BuildSim (`GET /api/sensors/{id}`) and publish it to the MQTT broker, so the pipeline and autonomous service can consume it. They emulate IoT devices that observe the environment and report it, they do not generate the values themselves.
- **Actuator processes** receive commands from the autonomous service and apply them to BuildSim (`PUT /api/actuators/{id}/state`); BuildSim is the single source of truth for actuator state.
- **The physical simulator** (Section 6) interacts *only* with BuildSim: it reads the current actuator state, advances the room physics, and writes the resulting sensor values back into BuildSim (`PUT /api/sensors/{id}/value`), this is what closes the feedback loop.
- **The dashboard** subscribes to BuildSim's WebSocket session API to highlight rooms and overlay the system's decisions on the 3D view.

Actors. The **building manager** monitors the system and may override it; **occupants** influence it only by being present, which the occupancy sensors detect. Neither actor is on the control path, the loop runs autonomously.

The scenario the demo tells. On a weekday morning, occupants arrive and a meeting fills room A2306. Body heat and exhaled CO_2 push temperature and CO_2 up. The autonomous service detects the rising CO_2 and the occupancy, increases ventilation in that zone, and holds the temperature in band, then, when the room empties, it falls back to a low-energy setback. Meanwhile unoccupied rooms are never heated or ventilated beyond the regulatory minimum. The same loop runs in every room, every cycle.

Why it matters. Buildings are among the largest energy consumers in Sweden, and ventilation and heating dominate that load. A controller that follows occupancy rather than the clock saves energy *and* improves air quality, but only if it is fast enough to react, robust enough to survive a failed sensor, and trustworthy enough to leave running. Those three concerns, latency, resilience, trust, drive the requirements in Section 3.

Where this system sits. Ventilation control is only one of several systems that run on the same building. The C4 *System Landscape* below shows the bigger picture, fire detection, security, energy management, and ours are each built on BuildSim and overseen by the facilities team. This report designs the highlighted system.



C4 System Landscape. Several independent building systems, each on BuildSim; the highlighted system is the one this report designs.

3. Requirements

The requirements below are the root of the MBSE thread: each is a **testable** statement with a unique ID, traced to a design element (Sections 4–9) and a verification (Section 10). Functional requirements (FR) say what the system does; non-functional requirements (NFR) say how well; the regulatory requirement (REG) comes from the Swedish Work Environment Authority's ventilation guidance. Vague requirements cannot be tested, so every one here is quantified.

ID	Type	Requirement	Priority	Acceptance criterion	Verified by
FR-01	Functional	Keep an occupied room's CO_2 below 1000 ppm by ventilating	Must	CO_2 returns below 1000 ppm within 10 min of exceeding it while occupied	`test_co2_recovery`
FR-02	Functional	Hold occupied rooms within the comfort band 20–24 °C	Must	In band $\geq$ 90 % of occupied minutes	`test_comfort_band`

ID	Type	Requirement	Priority	Acceptance criterion	Verified by
FR-03	Functional	Set back ventilation and heating in empty rooms	Must	Fan and setpoint drop within 15 min of a room emptying	`test_setback`
NFR-01	Non-functional	Produce a control decision at least every 60 s per zone	Must	Median decision interval $\leq$ 60 s under full load	`test_decision_interval`
NFR-02	Non-functional	Survive a sensor-process crash without losing control	Must	A new reading reaches storage within 60 s of a kill	`test_sensor_recovery`
NFR-03	Non-functional	Avoid actuator thrashing	Should	No actuator changes state more than once per 5 min	`test_no_thrashing`
NFR-04	Non-functional	Use less energy than a fixed-schedule baseline	Should	$\geq$ 15 % lower modelled energy over one week	`test_energy_saving`
REG-01	Regulatory	Maintain a minimum ventilation rate even when empty	Must	Fan never drops below the configured minimum	`test_min_ventilation`

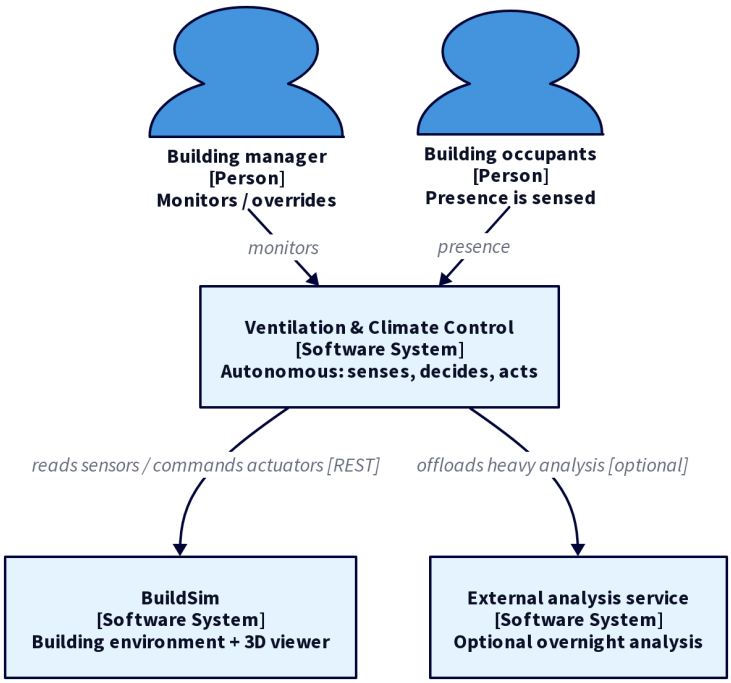
A deliberate **tension** runs through these requirements, air quality and comfort pull ventilation up (FR-01, FR-02), energy pulls it down (NFR-04), and the regulatory floor (REG-01) sets a hard minimum. Resolving that tension safely is the whole job of the autonomous service, and it is why a single threshold rule is not enough.

Traceability. The five architecture viewpoints map onto this document so nothing is left unverified: *context* (4.1), *functional/container* (4.2–4.3), *information/data* (6–7), *behavioural* (8), *deployment* (9). Each requirement names the design that satisfies it and the test that proves it; the plan in Section 10 closes the loop.

4. Architecture

4.1 Context diagram (C4 Level 1)

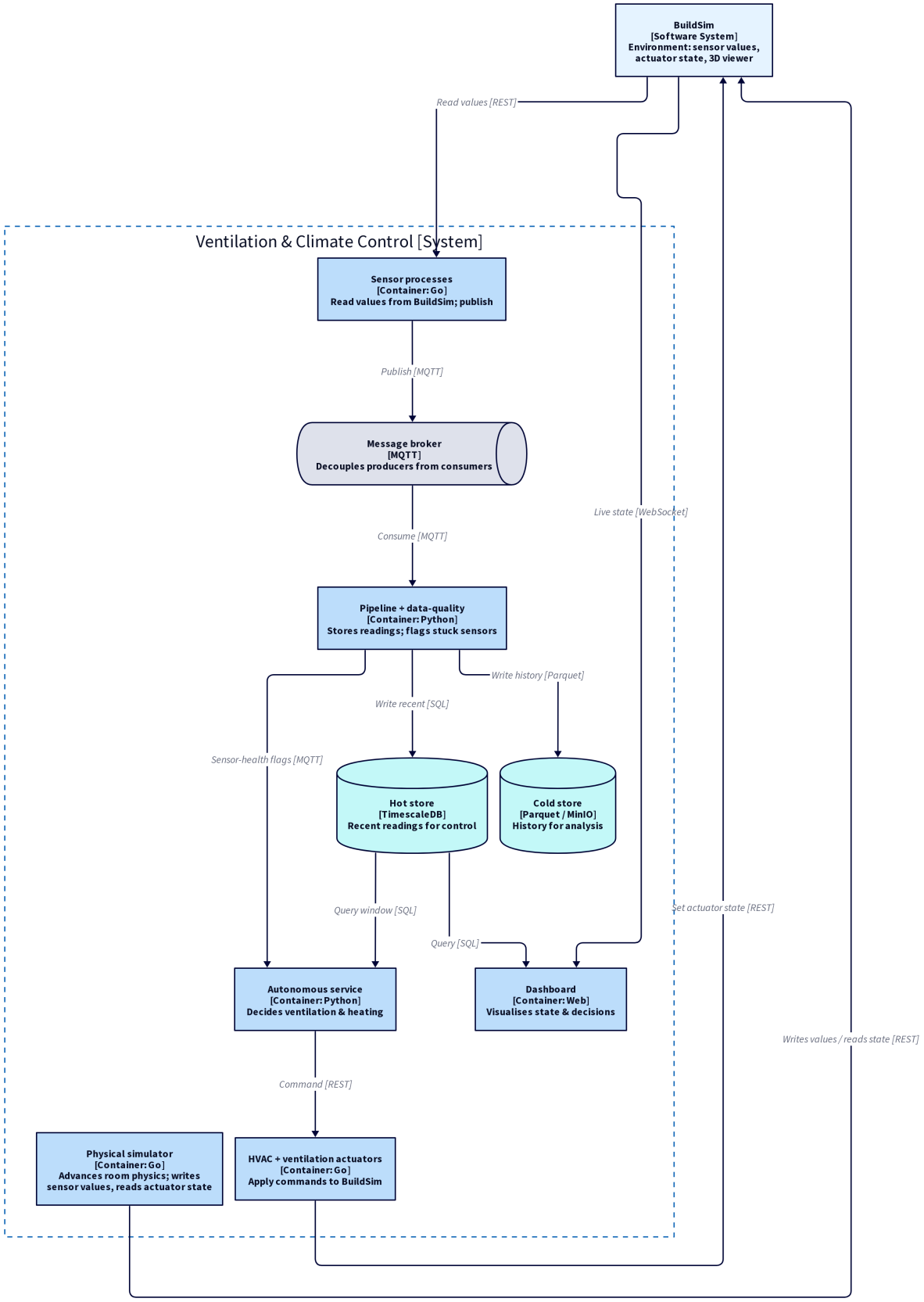
The context view fixes the **system boundary**: what is ours to build versus what is outside. The system reads and commands the building only through BuildSim; the building manager monitors and may override; occupants are sensed, not connected; and an optional external analysis service is used only for overnight analysis, never on the control path.



C4 System Context (Level 1). Person and software-system elements; the system in scope is highlighted, external systems are grey; every arrow is a directed relationship labelled with its purpose and protocol.

4.2 Container diagram (C4 Level 2)

Each box is one Docker container and one repository folder; the arrows carry the protocol. **BuildSim is the environment** and the system's shared state. The physical simulator interacts *only* with BuildSim: it reads the current actuator state, advances the room physics, and writes the resulting sensor values back into BuildSim. The sensor processes then read those values from BuildSim and publish them onto the broker; the autonomous service decides and commands the actuator processes, which write actuator state back to BuildSim. That actuator change alters the environment the simulator advances next tick, the closed loop that makes this a cyber-physical system rather than a dashboard.



C4 Container diagram (Level 2). Each container shows its [technology] and responsibility; the in-scope system is the dashed boundary, external systems (BuildSim) are grey; every arrow is a directed relationship labelled with its protocol.

4.3 Component responsibilities

Every container has a single responsibility, the principle that makes the system testable and lets one part fail without taking down the rest. The "reason for separation" column is where that systems-level reasoning is made explicit.

Component	Responsibility (one job)	Container	Tech	Reason for separation
Temperature sensor	Read simulated temp, publish	`temp-sensor`	Go	One per room; independent restart
CO_2 sensor	Read simulated CO_2, publish	`co2-sensor`	Go	Same lifecycle, different signal
Occupancy sensor	Publish presence / count	`occupancy-sensor`	Go	Drives both control and features
Physical simulator	Compute room physics from actuator state	`simulator`	Python	Models the building; closes the loop
MQTT broker	Decouple publishers from consumers	`mosquitto`	Mosquitto	One source, many consumers
Pipeline consumer	Persist readings (hot + cold)	`pipeline`	Python	Decouples ingestion from sensing
Data-quality monitor	Detect stuck / stale sensors	`dq-monitor`	Python	Second technique; protects the controller
Autonomous service	Decide ventilation and heating	`control-service`	Python	Decision logic evolves independently
HVAC / vent actuators	Apply commands to BuildSim	`*-actuator`	Go	Separate failure domains
Dashboard	Visualise state and decisions	`dashboard`	Web	Separates presentation from logic

4.4 Design decisions and justifications

This table carries most of the architecture grade. Each row states the option chosen, the alternative genuinely considered, the requirement it serves, and the trade-off accepted. A decision with no rejected alternative is only an assumption.

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Sensor transport	MQTT pub/sub	Direct REST; Kafka	Three consumers need the same readings; broker buffers if one is down (NFR-02)	A broker to run and monitor
Storage split	TimescaleDB (hot) + Parquet/MinIO (cold)	Single relational DB; files only	Fast recent-window queries for control, cheap history for analysis (NFR-01, NFR-04)	Two stores to keep consistent
Control approach	Rule-based + hysteresis + setback	ML controller; MPC optimiser	Meets the 60 s budget, is verifiable, matched ML on the available data (NFR-01)	Less adaptive than a learned controller
Data quality	Dedicated stuck-sensor monitor	Trust raw readings	A frozen sensor is a safety risk, not just noise (FR-01)	Extra component and tuning
Decomposition	One container per role	Monolith	Fault isolation and independent restart (NFR-02)	More inter-process communication
Heavy analysis	Overnight, on external compute	Real-time, on the laptop	Keeps the control loop fast; analysis is off the critical path (NFR-01)	Occupancy prior updates daily, not live

5. Interfaces and communication

Three communication patterns are used, each for the job it fits. **MQTT pub/sub** carries sensor readings because one sensor feeds many consumers and the broker buffers if a consumer is

briefly down. **REST** carries device registration and actuator commands, including every call to BuildSim, because each is a one-to-one request whose result the caller must confirm. **WebSocket** (BuildSim's) pushes live state to the dashboard without polling. Specifying these precisely is what would let another team rebuild the system from this section alone.

Every reading is a JSON object with explicit units, so no consumer has to guess:

```
{
  "ts": "2026-09-24T09:14:00Z",
  "sensor_id": "co2-A2306",
  "room": "A2306",
  "type": "co2",
  "unit": "ppm",
  "value": 612
}
```

Interface	Direction	Protocol	Endpoint / topic	Notes
Simulator → BuildSim	write sensor values	REST	`PUT /api/sensors/{id}/value`	makes the reading visible in the 3D viewer
Sensor ← BuildSim	read value	REST	`GET /api/sensors/{id}`	the sensor samples the environment
Sensor → broker	publish	MQTT	`sensors/{room}/{type}`	QoS 1, every 5 s
Service → BuildSim	command	REST	`PUT /api/actuators/{id}/state`	200 ok; 409 if manually overridden
Simulator ← BuildSim	read state	REST	`GET /api/actuators`	closes the physical loop
Service → dashboard	publish	MQTT	`decisions/{room}`	carries a human-readable reason

Each decision message carries a **reason** string (e.g. `"CO2 740ppm rising, 5 occupants -> fan level 2"`) so that every action is auditable after the fact, a small choice with a large effect on whether the system can be trusted.

6. Simulating the sensor values (the physical model)

Because BuildSim provides the rooms but not their physics, **the physical simulator that generates realistic sensor values was built**. The aim is not a physically perfect model but one with enough structure, trends, daily cycles, and genuine responses to actuator commands, that the autonomous service reacts to meaningful data and the closed loop actually closes. The simulator reads the current actuator state from BuildSim each tick (5 s), updates each room's physical state, and writes the new sensor values back into BuildSim; the sensor processes then read those values and publish them onto the broker. A **physics-based** model is used rather than replaying recorded traces, because the sensor values must respond to the actuators, and that response to actuation is exactly what the control loop depends on.

Occupancy drives everything else, so it is modelled explicitly, but as a **density sampler conditioned on room type and time of day**, not as one building-wide curve. Each room type carries its own daily profile: **offices** fill from 07:00–09:00 and stay roughly steady until people

leave 16:00–18:00; the **fika (coffee) room** spikes around 11:30 and at the mid-morning and afternoon breaks; **conference rooms** fill during booked meeting slots (a person attends a couple per day); corridors and toilets see only brief transient use; and at night every room is empty. Concretely, each tick the simulator draws each room's head-count from a distribution conditioned on the room's type $\tau(r)$ and the time of day,

$$n_{r,t} \sim \text{Binomial}\big(N_r,\ \rho_{\tau(r)}(t)\big),$$

where N_r is the room's capacity and $\rho_{\tau(r)}(t) \in [0,1]$ is the type-specific occupancy fraction over the day, offices roughly steady 09:00–17:00, the fika room peaking at 11:30, conference rooms high only during meeting slots, and every type $\rho = 0$ at night. (Each of a room's N_r places is occupied independently with probability $\rho_{\tau(r)}(t)$, so the count is naturally bounded by capacity and has a mean of $N_r\,\rho_{\tau(r)}(t)$.) The simulator writes the resulting count $n_{r,t}$ into BuildSim, the occupancy session API for the 3D view and the occupancy sensor values, and that count is the input to the temperature term (body heat), the CO_2 term (exhalation), and the control logic.

A **richer alternative**, which could be implemented later if higher fidelity were needed, is an *agent-based* model: give each person a daily routine (arrive, work in their office, visit the fika room and colleagues, attend a couple of meetings, break for lunch, when a fraction leave the building and others go to the fika room, and leave in the evening) and move them room-to-room along **BuildSim's navigation graph** (`GET /api/graph/route`), which yields correlated building-wide flows and realistic transient corridor occupancy. The conditional density sampler was chosen because it is markedly **simpler to implement**, has no routing to get wrong, and already gives the autonomous service the structure it needs, room-by-room trends, the 11:30 fika spike, and busy meeting rooms.

Temperature uses a discrete heat-balance update per room:

$$T_{t+1} = T_t + \frac{\Delta t}{C}\big(k_{\text{loss}}(T_{\text{out}} - T_t) + q_{\text{occ}}\,n_t + q_{\text{hvac}}\,u_t\big)$$

where n_t is the occupancy count, u_t the HVAC setpoint command read back from BuildSim, k_{loss} the envelope loss, C the room's thermal mass, and $q_{\text{occ}} \approx 100$ W per person. Heating therefore raises temperature *gradually*, not instantly, which is exactly why the controller must anticipate rather than merely react.

CO_2 follows a mass-balance update driven by occupancy and ventilation:

$$C_{t+1} = C_t + \frac{\Delta t}{V}\big(g\,n_t - Q_{\text{vent}}(v_t)\,(C_t - C_{\text{out}})\big)$$

with generation $g \approx 0.005$ m³/s per person, outdoor baseline $C_{\text{out}} = 420$ ppm, and ventilation flow Q_{vent} a function of the commanded fan level v_t. Opening the damper (raising v_t) pulls CO_2 back toward outdoor levels; an empty, unventilated room decays slowly to baseline.

The crucial property is that **actuator commands feed back into these equations through BuildSim**: when the controller raises the fan, Q_{vent} rises, CO_2 falls, and the next sensor reading reflects it. That is the sense–compute–act loop made concrete, and it is what lets the controller be evaluated against a baseline in Section 11. The model is intentionally simple; its limitations (no inter-room air exchange, idealised mixing) are listed honestly in Section 13.

7. Autonomous services and data engineering

7.1 The autonomous service

The autonomous service is the brain. It is **rule-based with hysteresis and an occupancy-aware setback**, and runs once per zone every 30 s.

Why this approach. The decision must land inside the 60 s budget (NFR-01), must be easy to verify for a safety-relevant function, and, on two weeks of data, a learned model gave no measurable benefit over a transparent rule set. Rules also make the regulatory floor (REG-01) a trivial hard clamp that no optimiser can override. This is the course's point in practice: the *simplest approach that meets the requirement* is the right one, and sophistication is not the goal.

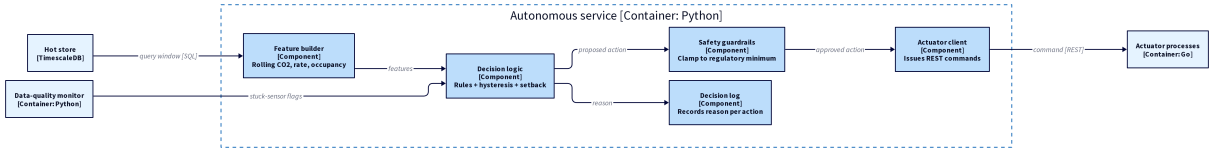
Decision logic. For each zone the service emits a heating setpoint and a ventilation level (0–3):

- **Occupied:** target 21 °C; raise the fan one level per 200 ppm of CO_2 above 700, clamped to level 3; never below the regulatory minimum.
- **Empty > 15 min:** drop to a 17 °C setback and minimum fan.
- **Hysteresis:** only change a setting if it differs *and* ≥ 5 min have passed since the last change (NFR-03), which prevents the oscillation a naïve threshold rule would cause.

Two techniques, combined. A purely rule-based controller has one blind spot found in testing (Section 11): a *stuck occupancy sensor reading "empty"* defeats the CO_2 response. A second, statistical technique was therefore added, the **data-quality monitor** (7.2), whose stuck-sensor flag the controller consumes: if a zone's occupancy sensor is flagged stale but its CO_2 is rising, the controller assumes the room is occupied and ventilates. Combining a rule controller with a statistical sensor-health check, and treating data quality as a first-class control input, is what lifts this from "it works" to a system that handles its own failure modes.

Failure and safety. If the service crashes, actuators **hold their last state** (they do not fail to "off"); a watchdog restarts it, and on restart it re-reads current state from BuildSim before deciding. The regulatory minimum is a hard clamp below all rules.

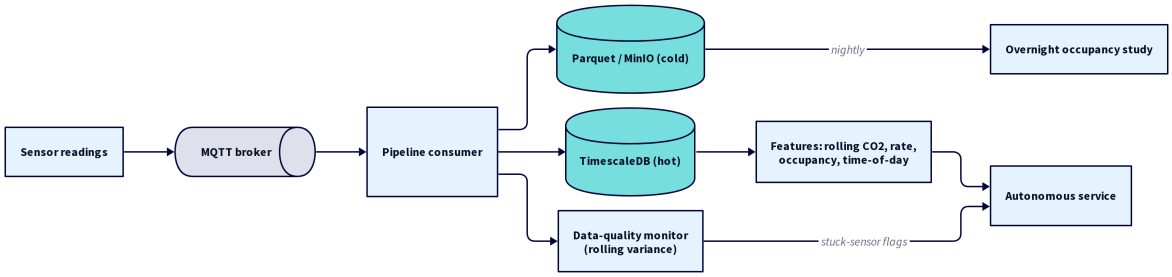
Inside the service (C4 Level 3). The autonomous service is the one container complex enough to warrant a component view. It builds features, applies the decision logic, passes every proposed action through the safety guardrails *before* any command leaves the service, and logs the reason for each action.



C4 Component diagram (Level 3), inside the Autonomous service container. Components are light blue, with their [technology] and a one-line responsibility; the guardrail sits between every decision and the actuator client.

7.2 The data pipeline

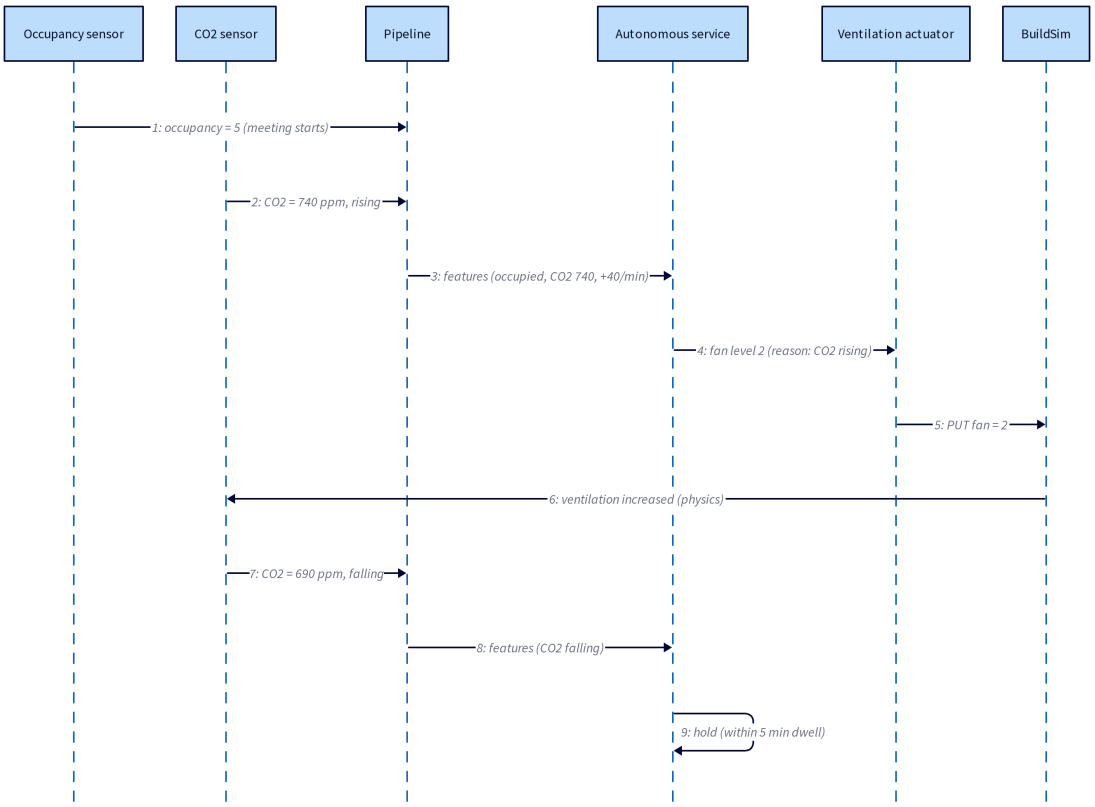
Readings travel through a small but real pipeline so the same data serves live control, the dashboard, the data-quality monitor, and overnight analysis. The broker buffers if storage is briefly slow (supporting NFR-02). The **hot path** (TimescaleDB) answers the controller's recent-window queries; the **cold path** (Parquet on MinIO) keeps immutable history for the energy analysis. A stream processor computes the features the controller consumes, rolling-mean CO_2, CO_2 rate-of-change, a smoothed occupancy estimate, and a cyclical time-of-day encoding, and the **data-quality monitor** computes a rolling variance per sensor: a variance of (near) zero over five minutes means the sensor is stuck, which it flags to the controller.



8. Behaviour

8.1 Key scenario, dynamic diagram

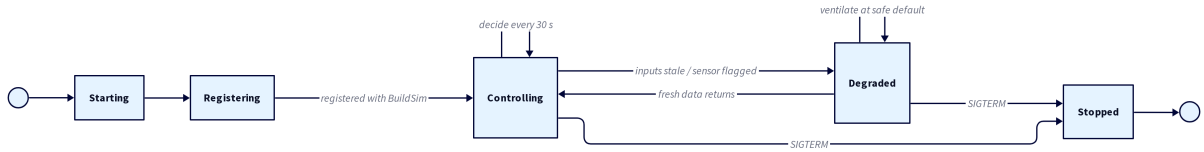
This is a C4 *dynamic* diagram: it shows how the containers from Section 4.2 collaborate at runtime for one scenario (the meeting from Section 2), with **numbered** interactions giving the order. It exposes the timing the static diagrams cannot, the controller acts on the *trend*, and the hysteresis rule then suppresses a needless second change.



C4 Dynamic diagram, numbered runtime interactions among the containers for the meeting scenario.

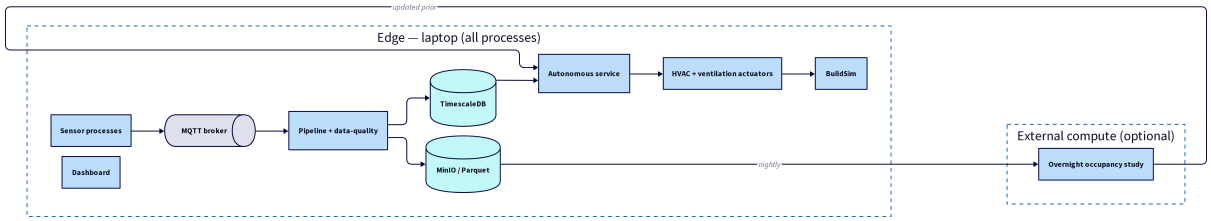
8.2 Process lifecycle, state machine

The autonomous service's own lifecycle is where the fault handling lives: it drops to a **degraded** state when its inputs go stale, ventilating at a safe default rather than trusting bad data, and recovers automatically.



9. Deployment and component placement

Every component runs as its own container on a single **laptop**, so deployment here is about how the system is composed from independent processes rather than geography. Each process is independently startable and restartable, communicates only through the interfaces of Section 5, and is isolated so that one failure does not cascade, if the dashboard crashes, control continues (NFR-02). The only *external* compute is the **overnight occupancy study**, deliberately off the critical path: if the external service is unreachable, the controller simply keeps yesterday's occupancy prior, so the building keeps regulating itself. Even with everything on one machine, the design keeps the *edge–cloud continuum* in view: the control loop (sensors, broker, storage, autonomous service, actuators) is the part that would run on a local edge server in a real deployment so it survives a network outage, while the slow occupancy study is the only part that would move to external compute.



10. Test plan

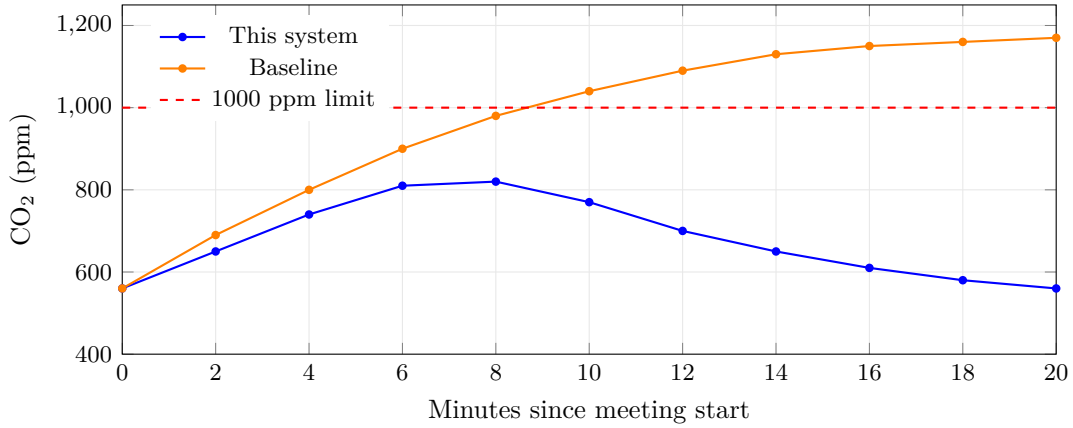
Every requirement has a verification approach and a concrete piece of evidence; components are tested in isolation, the full loop end-to-end, and, most informatively, under failure. This table is the validation half of the MBSE thread that began in Section 3.

Requirement	Test approach	Evidence
FR-01	Inject a CO_2 rise in an occupied room, watch ventilation respond	Time-series of CO_2 and fan level (§11)
FR-02	Simulate an occupied day, log temperature	% of occupied minutes in band
FR-03	Empty a room, observe setback	Setpoint/fan trace after vacancy
NFR-01	Scale the number of zones, measure decision interval	Latency-vs-zones curve (§11)
NFR-02	`docker kill` the CO_2 sensor mid-run	Recovery-time measurement
NFR-03	Replay noisy data, count actuator changes	Switch-count per 5 min
NFR-04	One-week run vs. fixed-schedule baseline	Modelled energy, both (§11)
REG-01	Force "empty building", inspect fan	Fan never below minimum (log)

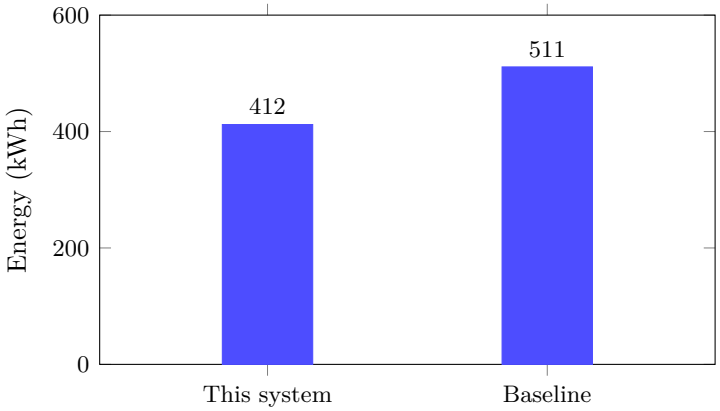
11. Evaluation and experiments

The system was evaluated over a simulated office week and against a **fixed-schedule baseline** that ventilates and heats on office hours regardless of occupancy. The point of this section is evidence, not assertion.

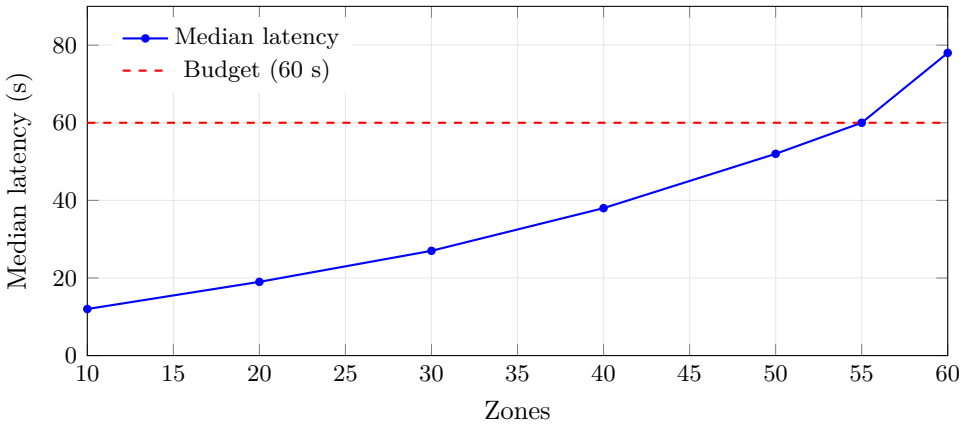
Air quality (FR-01). During a five-person meeting in A2306, the controller raised ventilation as CO_2 climbed and held it under the 1000 ppm limit; the fixed-schedule baseline, blind to the extra load, drifted past it. The closed loop of Section 6 is what produces the turn-around.



Energy (NFR-04, sustainability). Over the week the system used **412 kWh against the baseline's 511 kWh, a 19 % saving**, almost entirely from setback in unoccupied rooms. The small comfort penalty (93 % vs 96 % of occupied minutes in band) is the cost of letting empty rooms drift before someone arrives, and is exactly the energy-vs-comfort trade-off the requirements set up.



Scaling and the breaking point (NFR-01). The single-threaded controller held the 60 s budget up to **40 zones** and crossed it at about **55 zones**; the bottleneck is the per-zone TimescaleDB query, not the rules. Knowing *where* the system breaks, and *why*, is the difference between claiming it works and understanding it.



Headline results.

Metric	This system	Baseline	Requirement
Occupied minutes in comfort band	93 %	96 %	≥ 90 % (FR-02) ✓
CO_2 exceedance (occupied min > 1000 ppm)	0.6 %	4.1 %	as low as possible (FR-01) ✓
Modelled weekly energy	412 kWh	511 kWh	≥ 15 % saving → **19 %** (NFR-04) ✓
Median decision interval (≤ 40 zones)	31 s	,	≤ 60 s (NFR-01) ✓
Sensor-crash recovery	22 s	,	≤ 60 s (NFR-02) ✓

Fault and stress experiments. Killing a CO_2 sensor mid-run, the zone fell back to "ventilate safely" within one cycle and recovered 22 s after restart, with no bad commands issued from the frozen value. With hysteresis disabled, noisy CO_2 made the fan switch up to eight times in five minutes; enabling it dropped that to at most once (NFR-03). The stuck-occupancy-sensor case, which a pure rule controller failed, is now caught by the data-quality monitor's cross-check.

12. Dashboard

The dashboard extends BuildSim's 3D viewer with a side panel showing, per room: live temperature, CO_2, and occupancy; the current setpoint and fan level; and a scrolling log of the autonomous service's **decisions with their reasons**. A building-wide tile compares today's energy against the baseline. The decision log is the most valuable feature: an operator sees not just *what* the system did but *why*, which is what makes an autonomous system trustworthy enough to leave running.

13. Risks and critical reflection

The system trusts occupancy more than it should: the data-quality cross-check mitigates a *stuck* sensor, but a slowly *drifting* one could still mislead it, and detecting drift is harder than detecting a frozen value, this is the first thing to add next. The controller is also single-threaded; sharding zones across worker processes would push the scaling limit well past 55 rooms and is a small change given the clean decomposition. Finally, the energy figure is **modelled, not measured against real hardware**, so it should be read as a relative comparison; the honest version of the headline is "19 % less than this baseline, in simulation", not "19 % less energy".

Risk / assumption	Impact	Mitigation / decision
Occupancy sensor drifts (not just stuck)	Under-ventilation despite people present	Add drift detection (planned)
MQTT broker is a single point of failure	Sensor data stops flowing	Persistent sessions; restart policy; sensors buffer briefly
Simulated physics is optimistic	Energy/comfort figures flatter the system	Reported relative to a baseline, not absolute
Single-threaded controller	Caps building size at ~55 zones	Shard zones across workers if scaled